Téma 1: Zdroje dát

📖 Čo sa tu naučím

- Ako stiahnuť surový textový korpus z webu (requests + BeautifulSoup).
- Ako transformovať surovú term-frequency maticu (DTM) na binárnu, log-tf a IDF/TF-IDF reprezentáciu.
- Rozdiel medzi stemmingom a lematizáciou (NLTK WordNet).
- Ako spraviť POS-aware tokenizáciu, POS tagging a odstránenie stop slov + interpunkcie.
- Všetky vzorce vektorovej reprezentácie textu (binary, log frequency, IDF, TF-IDF).
- Aké sú **základné zdroje dát** pre objavovanie znalostí — databázy, text, web — a ako sa text prevádza do vektorovej podoby vhodnej pre dolovacie algoritmy.

🔑 Najdôležitejšie vzorce a koncepty

> **Z prednášky 1 (Proces objavovania znalostí):** Objavovanie znalostí (Knowledge Discovery) chápeme ako proces, ktorý zahŕňa **výber dát, predspracovanie dát, transformáciu dát, analýzu dát a interpretáciu výsledkov**. Rozlišujeme tri hlavné vetvy: *Knowledge Discovery in Databases*, *Knowledge Discovery from Texts (Text Mining)* a *Knowledge Discovery from Web (Web Mining)*. Pyramída hodnoty: dáta → informácie → znalosti → akčné znalosti.
>
> **Z prednášky 2 (Zdroje dát I — vektorová reprezentácia textu):** Text sa pre dolovanie reprezentuje vektorovo. Základné schémy: **word frequency (wf)**, **binary frequency** (`f(wf)=1` pre `wf>0`), **log frequency** (`f(wf) = 1 + log(wf)` pre `wf>0`) a **inverse document frequency (IDF)**, ktorá kombinuje výskyt v dokumente s rozšírenosťou v kolekcii: `tf-idf(t,d) = (1 + log(wf)) * log(N/df_t)` , kde `df_t` je počet dokumentov obsahujúcich termín `t` a `N` je počet dokumentov v kolekcii.
>
> **Z prednášky 3 (Zdroje dát II — transakčný/sekvenčný model):** Pri textových dátach okrem vektorovej reprezentácie existuje aj **transakčný/sekvenčný model** (každý dokument ako transakcia/sekvencia termínov, vhodné pre asociačné a sekvenčné pravidlá). Pri webe poznáme tri vetvy zdrojov: **obsah webu**, **mapa webu (štruktúra)** a **logovacie súbory** (záznam typu `IP - - [dátum] "GET /url HTTP/1.1" 200 15364 "referrer" "user-agent"`).

```
binary(t,d) = 1 ak tf(t,d) > 0, inak 0
log_tf(t,d) = 1 + log10(tf(t,d))  pre tf>0, inak 0
idf(t)       = log10(N / df_t)
tf-idf(t,d)  = log_tf(t,d) * idf(t)
```

- `N` = počet dokumentov v korpuse
- `df_t` = počet dokumentov obsahujúcich termín t (document frequency)
- IDF: zriedkavé slová → vysoká váha, časté slová ("the") → váha ≈ 0
- Lematizácia (WordNet) hľadá **skutočnú základnú formu zo slovníka** (presnejšia)
- Stemming (Porter) iba **oreže sufix** ("running" → "runn", rýchlejší ale hrubší)

Úloha 01: crawler.py

Teoretický základ: Web ako zdroj dát patrí do vetvy **Web Content Mining** — extrakcia zaujímavých a potenciálne užitočných znalostí z obsahu webových stránok. Prvým krokom procesu objavovania znalostí podľa metodiky CRISP-DM je **výber dát** (data selection) — práve to robí crawler: cielene zhromažďuje surový textový korpus, na ktorom následne bežia ďalšie fázy (predspracovanie, transformácia, analýza). *(viď prednáška 1 — Proces objavovania znalostí; prednáška 2 — Zdroje dát, sekcia Obsah webu)*

Čo robí: Stiahne 5 HTML stránok UKF FPVAI a uloží čistý text do `article_1.txt` .. `article_5.txt`.

Vstup → výstup: zoznam 5 URL (hardcoded) → `out/articles/article_{i}.txt` (UTF-8).

Kľúčové kroky: 1. `requests.get(url)` → HTTP GET, vracia Response objekt s `.text` (HTML string). 2. `BeautifulSoup(r.text, "html.parser")` → postaví DOM strom. 3. `soup.get_text()` → odstráni HTML tagy, ostanú len textové uzly. 4. `splitlines + strip + filter` → odstráni prázdne riadky. 5. `open(..., 'w', encoding='utf-8').write(text)` → uloží do .txt.

Dôležité parametre: - `encoding="utf-8"` — kvôli slovenskej diakritike (default Windows = cp1250 → UnicodeEncodeError). - `os.makedirs(..., exist_ok=True)` — opakovaný beh nehodí FileExistsError. - `enumerate(urls, 1)` — číslovanie od 1 (nie od 0), aby zodpovedalo "1. článok".

Časté otázky pri obhajobe: - **Q:** Prečo BeautifulSoup a nie regex? **A:** HTML nie je regulárny jazyk, regex je krehký. BS postaví DOM strom. - **Q:** Čo robí `.get_text()`? **A:** Prejde všetky textové uzly DOMu a skonkatenuje ich. HTML tagy sa vyhodia, obsah ostane. - **Q:** Prečo chýba `timeout=` a `r.raise_for_status()`? **A:** Audit minor — pre UKF web to funguje, anti-drift = nemeníme logiku. - **Q:** Do ktorej vetvy Web Miningu patrí crawlovanie textu? **A:** Web Content Mining — pracujeme s obsahom stránok, nie so štruktúrou ani s logmi. *(viď prednáška 1 — Proces objavovania znalostí)*

Úloha 02: vahove_schemy_tf_idf.py

Teoretický základ: Toto je jadro **vektorovej reprezentácie textu** z prednášky 2. Z jednej surovej term-frequency matice (word frequency, `wf`) odvodíme štyri alternatívne váhové schémy. Každá schéma má inú interpretáciu: **binary** zodpovedá Bernoulliho modelu (prítomný/neprítomný), **log frequency** tlmí extrémne časté slová logaritmickou transformáciou, **IDF** penalizuje slová prítomné vo veľa dokumentoch, a **TF-IDF** kombinuje obe — finálna váha je vysoká len pre slová časté lokálne a zriedkavé globálne. *(viď prednáška 2 — Vektorová reprezentácia textu, slide „inverse document frequency")*

Čo robí: Z vstupnej DTM (hárok "wf" v xlsx) vyrobí 5 hárkov: wf, binary, log, idf, tf-idf.

Vstup → výstup: `in/vektorova_reprezentacia.xlsx` (hárok "wf", tvar N×V) → `out/vektorova_reprezentacia.xlsx` (5 hárkov).

Kľúčové kroky: 1. `pd.read_excel(..., sheet_name="wf", index_col=0)` — načíta DTM, zachová názvy dokumentov ako index. 2. `binary = (wf > 0).astype(int)` — Bernoulli model. 3. `log_wf = np.where(wf>0, 1 + np.log10(wf, where=wf>0, out=zeros), 0)` — log-tf s ochranou pred log10(0)=-inf. 4. `doc_freq = (wf > 0).sum(axis=0)` — df_t pre každé slovo (sumuje cez riadky). 5. `idf = np.log10(N / np.where(df>0, df, 1))` — fallback proti deleniu nulou. 6. `tfidf = log_wf * idf` — broadcasting (N,V) * (V,). 7. `pd.ExcelWriter(...)` — multi-sheet zápis do jedného .xlsx.

Dôležité parametre: - `index_col=0` — prvý stĺpec → riadkový index (inak by sa zmenil tvar matice). - `sheet_name="wf"` — cieli presne na frekvenčný hárok (súbor môže mať viac hárkov). - `np.round(..., 4)` — 4 desatinné miesta pre čitateľnosť v Exceli.

Vzorce:

```
binary(t,d) = 1 ak wf>0, inak 0
log_tf      = 1 + log10(wf) pre wf>0, inak 0
idf(t)      = log10(N / df_t)
tf-idf      = log_tf * idf
```

Časté otázky pri obhajobe: - **Q:** Čo znamená IDF=0? **A:** Slovo je vo všetkých dokumentoch (df_t=N) → log10(N/N)=0 → potlačené (typicky stop slovo). - **Q:** Čo je broadcasting v `log_wf * idf`? **A:** log_wf má tvar (N,V), idf má (V,). Numpy roznásobí po stĺpcoch — každý riadok sa násobí tým istým idf vektorom. - **Q:** Prečo +1 v log-tf? **A:** Aby tf=1 dostalo váhu 1 (log10(1)=0 by neumožnilo odlíšiť "neprišlo" vs "1× prišlo"). *(viď prednáška 2 — log frequency: `f(wf) = 1 + log(wf)` pre `wf > 0`)* - **Q:** Ako sa volá schéma `(1 + log(wf)) * log(N/df_t)`? **A:** TF-IDF — kombinácia log-frekvencie termínu v dokumente a inverznej dokumentovej frekvencie cez celú kolekciu. Je to štandardná schéma vektorovej reprezentácie textu pre dolovanie.

Úloha 03: lematizacia_textov.py

Teoretický základ: Predtým, než môžeme aplikovať vektorovú reprezentáciu, musíme z neštruktúrovaného textu vyrobiť **term-document maticu (TDM)**. To si vyžaduje **predspracovanie textu** — tokenizáciu na vety a slová, normalizáciu (lowercase) a redukciu morfologických variantov cez **lematizáciu** alebo stemming. Lematizácia hľadá skutočnú slovníkovú formu (lemmu), čo je presnejšie ako stemming. *(viď prednáška 2 — sekcia Text, kolekcie elektronických dokumentov)*

Čo robí: 5 anglických .txt súborov → term-document matica lem → 4 váhové reprezentácie do jedného xlsx.

Vstup → výstup: `in/*.txt` (5 dokumentov) → `out/lemma_frekvencia.xlsx` (5 hárkov: wf, binary, log, idf, tf-idf).

Kľúčové kroky: 1. `nltk.download('punkt_tab' / 'wordnet' / ...)` — stiahne tokenizer + slovník. 2. `sorted(os.listdir("in"))` — deterministické poradie súborov. 3. `sent_tokenize(data)` — text → vety (Punkt model, vie "Dr. Smith."). 4. `word_tokenize(sentence)` — veta → tokeny. 5. `lemmatizer.lemmatize(word.lower())` — WordNet lematizátor (default POS='n'). 6. `dict.fromkeys(...)` — unikátne lemy v poradí prvého výskytu. 7. `wf.at[lemma, fname] += 1` — rýchly per-bunkový inkrement. 8. Odvodenie binary, log_tf, idf, tf-idf nad numpy poliami.

Dôležité parametre: - `encoding="utf-8"` — webové texty môžu mať non-ASCII (smart quotes). - `.lower()` pred lemmatize — case-insensitive ("Cat", "cat" → "cat"). - `idf[:, np.newaxis]` — z (L,) sprav (L,1) pre broadcasting s (L,N).

Vzorce: rovnaké ako úloha 02.

Časté otázky pri obhajobe: - **Q:** Rozdiel lematizácia vs stemming? **A:** Lematizácia hľadá slovníkovú formu (WordNet), stemming iba oreže sufix. Lematizácia je presnejšia, stemming je rýchlejší. - **Q:** Prečo dict.fromkeys a nie set? **A:** Zachová poradie prvého výskytu → reprodukovateľné stĺpce. - **Q:** Bug v skripte? **A:** Lematizuje bez POS argumentu → všetko sa berie ako noun ("running" ostane "running"). Úloha 04 to opravuje cez `get_wordnet_pos()`. - **Q:** Čo je term-document matica (TDM)? **A:** Riadky=termy (lemy), stĺpce=dokumenty. Hodnota = počet výskytov. Je to štandardný vstup pre vektorovú reprezentáciu textu. *(viď prednáška 2 — Vektorová reprezentácia textu)*

Úloha 04: pos_tagging_a_lematizacia.py

Teoretický základ: POS (Part-of-Speech) tagging priraďuje každému slovu jeho slovný druh na základe **syntaktického kontextu celej vety**. To umožňuje presnejšiu lematizáciu (napr. "better" ako prídavné meno → "good") a v textovom dolovaní sa využíva aj na filtráciu — typicky **podstatné mená (substantíva)** nesú najviac sémantickej informácie. *(viď prednáška 3 — Substantíva, odkaz na korpusovú lingvistiku: tokenizácia, garabik-gianitsova-horak-simkova)*

Čo robí: Z 5 .txt súborov vyrobí tabuľku 7 stĺpcov: `Subor, ID_veta, Slovo, Tag, Lemma, Poradie, Slovny_Druh`.

Vstup → výstup: `in/*.txt` → `out/identifikacia.xlsx` (~2060 záznamov).

Kľúčové kroky: 1. `sent_tokenize` → vety; `word_tokenize` → slová s interpunkciou. 2. `nltk.pos_tag(words)` — POS tagging **na plnej vete** (pred filtrovaním, kvôli kontextu). 3. Filter interpunkcie: `all(c in string.punctuation for c in word)` — zachytí aj '`', "'", '--'. 4. Filter stop slov: word.lower() in stop_words `(set, O(1) lookup). 5. POS-aware lematizácia:` lemmatizer.lemmatize(word.lower(), pos=get_wordnet_pos(tag)) `. 6. Záznam riadku do` rows` (list of dicts) → DataFrame → .to_excel().

Dôležité parametre: - `set(stopwords.words('english'))` — set kvôli O(1) membership testu. - `enumerate(sentences, 1)` — ID_veta od 1. - POS mapovanie: V→*v*, J→a, R*→r, ostatné→n (default).

Vzorce: žiadne (NLP pipeline).

Časté otázky pri obhajobe: - **Q:** Prečo POS tagging PRED filtrovaním? **A:** Tagger potrebuje syntaktický kontext (stop slová, interpunkcia). - **Q:** Prečo filter `all(c in punctuation for c in word)`? **A:** Zachytí aj viacznakové tokeny ('--', "'", '`') ktoré single-char in `test nepokryje. - **Q:** Prečo stĺpec Subor? **A:** ID_veta sa resetuje na 1 pre každý súbor – bez Subor by dvojica nebola unikátna globálne. - **Q:** Čo dáva POS-aware lematizácia oproti default? **A:** "using"+VBG → 'use', "better"+JJR → 'good'. Bez POS by ostali nezmenené. - **Q:** Čo je` Slovny_Druh`? **A:** Prvý znak Penn tagu — hrubá kategória: N=noun, V=verb, J=adj, R=adv. V textovom dolovaní sú najinformatívnejšie substantíva. *(viď prednáška 3 — Substantíva)*

🎯 Vyhodnotenie / na čo si dať pozor pri obhajobe

- **Proces KDD (5 fáz):** výber dát → predspracovanie → transformácia → analýza → interpretácia výsledkov. Pyramída hodnoty: dáta → informácie → znalosti → akčné znalosti. *(viď prednáška 1)*
- **3 vetvy Web Miningu:** Web Content Mining (obsah stránok), Web Structure Mining (mapa webu, hypertextové prepojenia), Web Usage Mining / Web Log Mining (logovacie súbory, správanie používateľov). *(viď prednáška 1 a 3)*
- **Zdroje dát (3 kategórie):** databázy (produkčné, dátové sklady, verejné ako DATAcube/STATdat/Eurostat), text (kvalifikačné práce, lekárske správy, dotazníky), web (obsah, mapa, logy). *(viď prednáška 2)*
- **Vzorce vedieť naspamäť:** binary, log-tf, idf, tf-idf — vrátane "prečo +1 v log-tf".
- **IDF interpretácia:** vysoké IDF = zriedkavé a diskriminačné slovo; IDF≈0 = časté/stop slovo.
- **N vs df_t:** N = počet dokumentov v korpuse; df_t = v koľkých dokumentoch je term t (sumuje cez RIADKY, axis=0).

- **Stemming vs lematizácia:** stemming oreže, lematizácia hľadá platnú formu v slovníku.
- **POS pred filtrovaním:** tagger potrebuje kontext; default WordNetLemmatizer = noun (bug úlohy 03).
- **Transakčný/sekvenčný model textu:** alternatíva k vektorovému — každý dokument ako množina/sekvencia termínov, vhodné pre asociačné/sekvenčné pravidlá. *(viď prednáška 3)*
- `encoding='utf-8'` **VŠADE:** Windows default cp1250 by hádzal UnicodeEncodeError na diakritike.
- `pd.ExcelWriter` **context manager:** kvôli viacerým hárkom v jednom xlsx + bezpečný flush.
- **Broadcasting (V,) * (N,V):** každý riadok dokumentu sa násobí rovnakým idf vektorom.